

FT1536

Full-Ternary-Secret Signatures over a Degree-1536 NTRU Ring

Design, Specification and Security Analysis

Niirmata

`niirmataa/free_falcon_sign`

Research specification — Draft 0.2
19 September 2026

This manuscript describes the FT1536 construction, its selected implementation profile, and the current security argument. The corrected verifier has source-bound formal proof checkpoints; end-to-end unforgeability and a concrete security level are not claimed. The M0 protocol wrapper is specified separately from the integrated C core.

Document snapshot: `f5266765c5f0733fb3ab6a5e2906aa44d15e17f7`

Integrated core: `2959064e8132443649de50600b20bfc32ed618cb`

Abstract

FT1536 is an experimental hash-and-sign lattice construction continuing the historical Falcon EXTRA ternary-ring implementation. It uses the ring $\mathbb{Z}[X]/(X^{1536} - X^{768} + 1)$, modulus 18433, and secret-polynomial proposals with coefficients in $\{-1, 0, 1\}$. The construction combines an NTRU trapdoor, mixed-radix Fourier sampling, an integer-emulated floating-point backend, and an adaptive five-level scalar proposal distribution. Its public verification equation is $z_1 + hz_2 = c$ modulo the ring and modulus, with a strict bound on an offset-768 quadratic form.

This document presents the design choices, algebraic conventions, key generation, signing, verification, encoding formats, and the exact M0 protocol contract. It separates the mathematical sampling target from the source algorithm and distinguishes the shared-key distribution generated by the actual seeded implementation from an ideal independent-coefficient law. It then develops the current security-analysis structure: byte-level extraction into MT-ISIS, image-distribution estimates, and a conditional success-event transfer based on χ^2 divergence.

Established repository results include canonical normalization for all signed 16-bit inputs; full modular-transform composition; extraction of a short lattice witness from every payload accepted by the modeled corrected verifier; and a STATIC payload bound of 3160 bytes after defined norm acceptance. A synthetic short vector requiring 3156 bytes motivates the M0 capacity of 4096 bytes. The corrected core is integrated, whereas M0's fixed-40-byte-nonce wrapper remains separate. Formal proof artifacts, source-model correspondence, and reported execution checks are documented independently. The remaining signing-law and reduction obligations are stated explicitly so that later results can extend the specification without changing the meaning of earlier theorems.

Document conventions

The exposition is organized around the construction rather than the chronology of its proof tasks. Main algorithms give the protocol and control-flow contracts. Bit-exact table contents and low-level FPEMU operations are tied to the pinned implementation; this draft is not a replacement for those source files. Statements labeled as established refer to the cited repository checkpoints and their declared models. Analytical lemmas proved here are distinguished from their still-pending instantiation in FT1536.

This is an English working manuscript prepared for the project author's technical review. No institutional affiliation, submission venue, or third-party certification is asserted. The original Falcon authors retain credit for the inherited design and implementation lineage. The title does not imply that FT1536 is an official Falcon parameter set.

Contents

Abstract	i
Algorithms	iv
1 Introduction and Design Rationale	1
1.1 Construction overview	1
1.2 Three meanings of ternary	1
1.3 Design choices and their consequences	1
1.4 The selected profile	2
1.5 What is fixed, proved, and still being analyzed	2
2 Algebraic and Probabilistic Foundations	3
2.1 The ring and coefficient convention	3
2.2 Canonical, centered, and signed representatives	3
2.3 The quadratic form	4
2.4 NTRU keys and the public relation	4
2.5 Fourier and number-theoretic representations	5
2.6 Reference sampling distributions	5
3 Specification of FT1536	6
3.1 Algorithm layers and interfaces	6
3.2 Key-pair generation	6
3.3 Target derivation and nonce framing	8
3.4 Trapdoor preparation and Fourier sampling	8
3.5 Adaptive scalar sampling	9
3.6 Signature generation	10
3.7 Signature verification	11
3.8 Functional correctness and the ideal construction	11
4 Encoding Formats and Protocol Parameters	12
4.1 Records, headers, and byte lengths	12
4.2 Public and private keys	12
4.3 STATIC encoding	13
4.4 Accepted STATIC and NONE decoding	13
4.5 A sufficient bound on honest payload capacity	14
4.6 Transport and integration requirements	14
5 Security Analysis	15
5.1 The target experiment	15
5.2 The natural MT-ISIS relation	15

5.3	Established verifier-to-witness soundness	16
5.4	Signing distributions and image estimates	16
5.5	From scalar approximation to the whole signing oracle	16
5.6	A direct success-event transfer	17
5.7	The reduction architecture	18
5.8	Cryptanalytic questions and parameter assessment	18
6	Implementation and Evaluation	19
6.1	Reference source and build selection	19
6.2	FPEMU and the scalar tables	19
6.3	Modular verification pipeline	19
6.4	Space and size results	20
6.5	Reported validation results	20
6.6	Performance reporting policy	20
6.7	Reproducibility and the trust boundary	21
7	Research Status and Further Development	22
7.1	Results established at this snapshot	22
7.2	The remaining dependency structure	22
7.3	Specification changes versus proof extensions	22
7.4	Conclusion	22
A	Selected Proofs and Consistency Arguments	23
A.1	Canonical normalization on the full signed16 domain	23
A.2	Verifier soundness from the completed pipeline	23
A.3	Integer-only derivation of the capacity endpoint	23
A.4	Why fixed nonce length is enough for input framing	24
A.5	Interpreting the loss comparison	24
B	Artifact Map and Manuscript Boundaries	25
B.1	Source identity	25
B.2	Reader’s map to the evidence	25
B.3	Formal results and unproved target interfaces	26
B.4	Attribution and preparation of this draft	26
B.5	Editorial completion before submission	26
	Bibliography	27

List of Algorithms

1	Coefficient-domain reference multiplication $\text{Mul}_\Phi(a, b)$	3
2	Ideal-bit description of a full-ternary coefficient proposal	6
3	Source-profile KeyGen, at the gate and serialization level	7
4	Distribution-level HashToPoint for FT1536	8
5	One signing attempt: source-level sampling interface	9
6	Adaptive scalar sampler: selected source control flow	9
7	M0 signing call using the corrected core	10
8	M0 verification through the byte-oriented corrected core	11
9	Bit-level STATIC encoding with $j = 8$	13

Chapter 1

Introduction and Design Rationale

1.1 Construction overview

FT1536 investigates a full-ternary-secret instantiation of a structured hash-and-sign signature. The public key is a polynomial h ; the secret key encodes a short NTRU basis; and a signature supplies a short response to a salted message target. Verification reconstructs the untransmitted first component and checks its length jointly with the transmitted second component. The implementation and the formal verification work are developed together in the FREE Falcon repository [1, 2].

The high-level architecture can be summarized as

$$\text{FT1536} = \text{NTRU relation} + \text{full ternary secret proposals} \\ + \text{mixed-radix sampling} + \text{explicit byte semantics}.$$

This identity is a description of the design, not a reduction theorem. The security task is to show that its actual signing distribution admits a suitable simulation and that every accepted adversarial byte string yields a witness for the exact hard relation.

Falcon provides the historical framework: it combines the GPV hash-and-sign approach, NTRU lattices, and fast Fourier sampling. Its version-1.2 specification records the earlier degree-768 ternary-ring parameter set and its removal from that version [3, 4]. FT1536 preserves the corresponding implementation lineage while changing the degree, secret proposal law, sampling profile, and acceptance parameters. Consequently, security claims for Falcon cannot simply be carried over.

1.2 Three meanings of ternary

The terminology must separate three different objects. A *ternary ring* here means the cyclotomic polynomial $X^N - X^{N/2} + 1$ with $N = 3 \cdot 2^9$. A *ternary secret* means that the raw coefficients of f and g take values in $\{-1, 0, 1\}$. Finally, the historical `full` flag selects a transform recursion shape. Neither the ring polynomial nor that flag implies a three-valued secret.

The current build selects `TRUE_TERNARY_SECRET=1` and mode 1. Its KeyGen replaces the earlier rounded-Gaussian-in-FFT3 proposal with coefficient-domain ternary proposals. This condition applies to f, g , not to the NTRU companions F, G , and not to the signature coefficients. The successful-key population includes all subsequent acceptance tests [5, 6].

1.3 Design choices and their consequences

1.3.1 A structured ring and a two-component witness

The public map $A_h(z_1, z_2) = z_1 + hz_2$ makes the first component recoverable from the target and the second. This permits transmitting only z_2 while checking the full pair. The ring supplies efficient multiplication, but also imposes algebraic structure that must remain visible in the cryptanalytic assumptions. In particular, replacing this public map with an unrelated random matrix would change the security problem.

1.3.2 Small secret proposals and an adaptive sampler

Moving f, g to three-valued coefficient proposals changes the geometry of accepted trapdoors. The current source explicitly notes that the resulting scalar widths can exceed the historical fixed proposal range. It therefore selects among five proposal families with nominal variances 5, 20, 80, 320, 768, using coefficients matched to the implementation’s floating-point representations [7]. This is a concrete engineering consequence of the secret-profile change, not evidence that the final signing law is already ideal Gaussian.

1.3.3 A norm that preserves the ring geometry

The verifier uses paired coefficients at offset 768, not a sum of independent coefficient squares alone. The cross terms are part of the specification. They affect both the acceptance region and the worst-case length of compressed signatures. A proof or benchmark using a different norm would concern a different profile.

1.3.4 An implementation-linked security interface

The intended reduction needs a bridge from the *actual accepted bytes* to a short witness. FT1536 treats normalization, modular transforms, signed narrowing, parser termination, and norm accumulation as components of that bridge. The earlier verifier’s normalization defect demonstrates why this is necessary. Its repair and the later proof chain are summarized in Chapter 5; details of the historical counterexample are retained in Appendix A [8, 9].

1.4 The selected profile

Property	FT1536 research profile
Degree and ring	$N = 1536, \Phi = X^{1536} - X^{768} + 1$
Modulus	$q = 18433$
Secret proposal	$f_i, g_i \in \{-1, 0, 1\}$ before KeyGen gates
NTRU companions	$F, G \in R$; emitted coefficients bounded by the source conversion
Signing target width	$\sigma = 768$
Strict squared-norm bound	$B = 2093922385$; acceptance is $Q < B$
Outer attempt limits	3,000,000 for KeyGen; 16 for Sign
Arithmetic	FPEMU for real/complex work; integer NTT for verification
Honest encoding	STATIC; accepted verification encodings: NONE and STATIC
M0 caller	40 nonce bytes; 4096-byte payload capacity
Current guarantee	Pinned-model verifier soundness and local capacity results
Security level	Not assigned by this draft

Table 1.1: Selected parameters and scope. Source profile and M0 caller are distinct layers [1, 6, 10].

1.5 What is fixed, proved, and still being analyzed

Three identities are maintained throughout this specification. The *historical S17* is an immutable reference with a recorded counterexample. The *corrected core* differs in its verifier normalization and is integrated in Extra/c. The *M0 caller* chooses a larger honest signing buffer and enforces a fixed nonce length, but is not integrated into the historical command-line wrapper [2, 11].

Chapter 2

Algebraic and Probabilistic Foundations

2.1 The ring and coefficient convention

Let

$$N = 1536, \quad m = 4608, \quad q = 18433, \quad \Phi(X) = X^N - X^{N/2} + 1 = \Phi_{4608}(X).$$

Write $R = \mathbb{Z}[X]/(\Phi)$ and $R_q = R/qR$. Each ring element is represented by a polynomial of degree below N . Arrays store coefficients in ascending degree order. A relation in R_q is not an equality of unreduced integer arrays: both reduction by Φ and reduction modulo q are required [2, 12].

The ring reduction identity is

$$X^N = X^{N/2} - 1 \quad \text{in } R.$$

For example, the coefficient of X^N contributes positively at degree $N/2$ and negatively at degree 0. Processing an ordinary product from its highest degree downward yields a simple coefficient-domain reference implementation. It provides a definition independent of the optimized transform.

Algorithm 1 Coefficient-domain reference multiplication $\text{Mul}_\Phi(a, b)$

Input: $a, b \in \mathbb{Z}^N$; modulus q

Output: Canonical coefficients of ab in R_q

```

1:  $u[0 \dots 2N - 2] \leftarrow 0$ 
2: for  $i = 0, \dots, N - 1$  do
3:   for  $j = 0, \dots, N - 1$  do
4:      $u[i + j] \leftarrow u[i + j] + a_i b_j$ 
5:   end for
6: end for
7: for  $k = 2N - 2, \dots, N$  in decreasing order do
8:    $t \leftarrow u[k]; u[k] \leftarrow 0$ 
9:    $u[k - N/2] \leftarrow u[k - N/2] + t$ 
10:   $u[k - N] \leftarrow u[k - N] - t$ 
11: end for
12: return  $(u_0 \bmod q, \dots, u_{N-1} \bmod q)$ 
```

Algorithm 1 is an exact-arithmetic reference, not a word-size C implementation. Its loop invariant is polynomial congruence modulo Φ . Descending order ensures that any newly created term of degree at least N is subsequently reduced. This definition has quadratic multiplication cost and is suitable for independent checks; it does not prescribe the optimized runtime path.

2.2 Canonical, centered, and signed representatives

For $x \in \mathbb{Z}$, define

$$\rho(x) = x \bmod q \in [0, 18432], \quad \text{center}_q(x) = ((x + 9216) \bmod q) - 9216.$$

The centered interval is $[-9216, 9216]$. A *canonical vector* lies in $[0, 18432]^N$; a *signed16 vector* lies in $[-32768, 32767]^N$. Both conditions are representation properties, not lattice assumptions. Since q is

odd,

$$\text{center}_q(-x) = -\text{center}_q(x), \quad \text{center}_q(x) \equiv x \pmod{q}.$$

Normalization before multiplication must not replace the original signed vector in the norm. A coefficient such as -20000 has canonical residue 16866 , but these representatives have different integer magnitudes. The verifier uses the residue for modular arithmetic and the decoded signed value for the second component's norm [13, 14].

2.3 The quadratic form

For $a \in \mathbb{Z}^N$, write

$$Q_0(a) = \sum_{i=0}^{767} \left(a_i^2 + a_i a_{i+768} + a_{i+768}^2 \right), \quad (2.1)$$

$$Q(a, b) = Q_0(a) + Q_0(b). \quad (2.2)$$

Every paired block has matrix $\begin{pmatrix} 1 & 1/2 \\ 1/2 & 1 \end{pmatrix}$. Its eigenvalues are $1/2$ and $3/2$, so

$$\frac{1}{2} \|a\|_2^2 \leq Q_0(a) \leq \frac{3}{2} \|a\|_2^2.$$

Also,

$$2(x^2 + xy + y^2) = x^2 + y^2 + (x + y)^2.$$

These identities show positive definiteness and explain why the negative cross terms cannot be discarded. In particular $Q_0(-a) = Q_0(a)$, although changing only one coordinate in a pair generally changes the norm. The acceptance threshold is strict: for integer Q , $Q < B$ means $Q \leq B - 1$ [12, 15].

2.4 NTRU keys and the public relation

The public key is

$$h = g/f \quad \text{in } R_q,$$

where KeyGen requires f to be invertible modulo (Φ, q) . The secret companions satisfy the NTRU equation

$$fG - gF = q \quad \text{in } R.$$

Using row-vector conventions, define

$$\mathcal{B} = \begin{pmatrix} g & -f \\ G & -F \end{pmatrix}, \quad A_h(z_1, z_2) = z_1 + h z_2.$$

Both rows of $\mathcal{B}$ are mapped to zero modulo q . Thus their integer coefficient combinations produce vectors in

$$\Lambda_h = \{(z_1, z_2) \in R^2 : A_h(z_1, z_2) = 0 \text{ in } R_q\}.$$

The map A_h is surjective because $(c, 0)$ maps to any $c \in R_q$; hence its kernel has index q^N in the coefficient lattice $\mathbb{Z}^{2N}$. The exact NTRU equation is the trapdoor identity used in the following idealized signing description [5, 10].

For a target c , a convenient rational-coordinate lift is

$$t = (-cF/q, cf/q), \quad t\mathcal{B} = (c, 0).$$

If an integer ring pair y is sampled near t and $w = y\mathcal{B}$, then $z = (c, 0) - w$ satisfies $A_h(z) = c$. Producing a short z with the right distribution is the sampling problem. The relation alone says nothing about leakage of the secret basis.

2.5 Fourier and number-theoretic representations

Two transforms have different jobs. FFT3 evaluates over complex roots, represented through FPEMU, and supports Gram computations and sampling. The NTT evaluates in $\mathbb{F}_q$ and supports exact modular multiplication. Their implementations share mixed-radix structure but not a proof of their numerical behavior.

Here $q - 1 = 18432 = 4 \cdot 4608$, so $\mathbb{F}_q^\times$ contains primitive 4608th roots. Evaluating at the N distinct roots of Φ gives a ring map to $\mathbb{F}_q^N$, under which multiplication is componentwise. The implementation fixes a particular physical order, indexed by $3i + j$ with $0 \leq i < 512$, $0 \leq j < 3$. Its source-bound forward theorem identifies this order rather than merely establishing invertibility [16, 17].

The recurrence combines a first quadratic split, eight binary levels, and a final cubic stage. Splitting a polynomial into even and odd coefficients is an algebraic identity; maintaining the correct child moduli, root labels, and memory positions is the implementation-specific part. This is why an arbitrary mutually inverse transform pair would not suffice to justify multiplication.

2.6 Reference sampling distributions

Let $\sigma = 768$ and define the ambient distribution on $R^2 \cong \mathbb{Z}^{3072}$ by

$$D(z) = \frac{\exp(-Q(z)/(2\sigma^2))}{\sum_{w \in \mathbb{Z}^{3072}} \exp(-Q(w)/(2\sigma^2))}.$$

Write $D^B = D \mid \{Q < B\}$. For a public key h and a target c , the reference coset distributions are $D \mid \{A_h = c\}$ and, when the event has positive mass, $D^B \mid \{A_h = c\}$. These are mathematical objects. They are not definitions of the actual output of `sampler_large`, `FPEMU`, or `Sign` [10].

The GPV framework motivates basis-independent preimage sampling [4]. In FT1536, the required source-to-reference statement must additionally cover finite proposals, computed scalar widths, transform errors, integer casts, retry limits, and the observable failure behavior. Defining an ideal law is only the beginning of that comparison.

Specification of FT1536

3.1 Algorithm layers and interfaces

The specification distinguishes three layers: exact ring algorithms; the selected, source-linked arithmetic and control flow; and the M0 caller that supplies framing and capacities. A mathematical expression such as $1/(2\tau^2)$ denotes exact arithmetic unless decorated with fl . The latter denotes the sequence of FPEMU operations in the pinned source, not an arbitrary evaluation of the same real expression [7, 10].

The three main interfaces are KeyGen , $\text{Sign}(sk, m)$, and $\text{Verify}(pk, m, r, b)$. The signature record is (r, b) with exactly 40 nonce bytes. The honest payload uses STATIC and a 4096-byte caller capacity. Verification does not impose that honest capacity on an adversary’s payload. A byte length must be representable by the modeled API and refer to legal memory.

3.2 Key-pair generation

3.2.1 Full-ternary coefficient proposals

Mode 1 reads two random bits at a time, rejects one of the four values, and maps the remaining three to $-1, 0, 1$. With an ideal independent bit stream, the resulting coefficients are uniform on this set. With the actual fixed-length seed and deterministic expansion, the procedure defines a particular seeded law instead [5, 10].

Algorithm 2 Ideal-bit description of a full-ternary coefficient proposal

Input: A bit stream; length N

Output: $a \in \{-1, 0, 1\}^N$

```

1: for  $i = 0, \dots, N - 1$  do
2:   repeat
3:     Read a two-bit value  $u \in \{0, 1, 2, 3\}$ 
4:   until  $u \neq 3$ 
5:    $a_i \leftarrow$  the source bijection of  $u$  into  $\{-1, 0, 1\}$ 
6: end for
7: return  $a$ 
```

The pseudocode characterizes the proposal law; exact seeded replay also uses the source’s bit-consumption and value-order conventions. It does not resample successful keys to force an independent-coefficient interpretation after filtering.

3.2.2 Acceptance gates and NTRU solution

The current KeyGen first tests odd resultants modulo 2, then the raw and orthogonalized Fourier-domain norms, then public-key invertibility, NTRU solvability, and the final leaf certificate. The raw and orthogonalized tests use the selected FPEMU computations and the source threshold expression

$$\text{fl}\left(\frac{4Nq}{\sqrt{8}} \cdot \frac{1250}{100}\right).$$

This expression is a KeyGen filtering threshold, not the signature bound B , and the factor 12.5 is not the signing width [5, 6].

Algorithm 3 Source-profile KeyGen, at the gate and serialization level

Input: Selected FT1536 source profile; environment $\mathcal{E}$

Output: Encoded pair (sk, pk) , or failure

```

1: Allocate the KeyGen context and caller buffers; fail on service failure
2: Seed the source SHAKE state with 32 fresh entropy bytes
3: for  $a = 1, \dots, 3,000,000$  do
4:   Generate  $f, g$  with the full-ternary source proposal
5:   if either resultant test modulo 2 fails then
6:     continue
7:   end if
8:   Compute the raw and orthogonalized source norm tests
9:   if either test fails then
10:    continue
11:  end if
12:  if  $f$  is not invertible in  $R_q$  then
13:    continue
14:  end if
15:   $h \leftarrow g/f$  in  $R_q$ 
16:  Attempt the source NTRU solver and reduction for  $fG - gF = q$ 
17:  if solution, companion-range, or mandatory leaf checks fail then
18:    continue
19:  end if
20:  Encode  $f, g, F, G$  using the selected private-key format
21:  Encode the canonical public polynomial  $h$ 
22:  if either final serialization fails then
23:    return failure
24:  end if
25:  return  $(sk, pk)$ 
26: end for
27: return failure

```

The companion conversion rejects coefficients outside $[-2047, 2047]$ in the emitted-key path. The loader can recognize a broader set of encodings, so the emitted-key population must not be replaced with the entire loader-accepted population in signing proofs [5, 10, 18]. Inner sampling and solver behavior is tied to the source; the outer attempt limit does not introduce a new timeout for every inner loop.

The mathematical purpose of the NTRU solver is to obtain companions, not to change the public map. An elementary reduction replaces (F, G) by $(F - kf, G - kg)$ and preserves $fG - gF = q$. The selected implementation's recursion and reduction decisions fix the actual companion distribution. A different solver or a new rejection gate may preserve the NTRU equation while changing the successful-key law.

3.2.3 The successful-key distribution

For a fixed public service environment $\mathcal{E}$, let E_K denote success of the entire KeyGen call, including final serialization. If $p_K = \Pr(E_K) > 0$, define

$$\mathcal{K}_{\text{seed}}[\mathcal{E}] = \mathcal{L}((sk, pk) \mid E_K).$$

This pair is drawn once and reused throughout a security experiment. The actual seed expansion, all source gates, and the attempt cap are part of this law. The ideal infinite-tape law $\mathcal{K}_{\text{iid}}$ is a separate

reference [10].

For the corresponding experiment that terminates without a win on KeyGen failure,

$$\text{Adv}_{\text{uncond}} = p_K \text{Adv}_{\text{cond}}.$$

This identity applies once at setup. It does not produce a factor p_K for each signing query.

3.3 Target derivation and nonce framing

The concrete public hash path initializes SHAKE-256, injects r followed by m , and maps output words to coefficients. A 16-bit word is accepted if it is below $3q = 55299$, then reduced modulo q . Under an ideal independent uniform word stream, each residue has three preimages in the accepted interval; the resulting coefficient is exactly uniform [19, 20].

Algorithm 4 Distribution-level HashToPoint for FT1536

Input: Byte strings r, m , with $|r| = 40$ in M0

Output: $c \in [0, q - 1]^N$

- 1: Initialize SHAKE-256 and absorb $r \parallel m$
 - 2: $i \leftarrow 0$
 - 3: **while** $i < N$ **do**
 - 4: Read the next source-ordered 16-bit word w
 - 5: **if** $w < 55299$ **then**
 - 6: $c_i \leftarrow w \bmod 18433$; $i \leftarrow i + 1$
 - 7: **end if**
 - 8: **end while**
 - 9: **return** c
-

The direct-output ROM used in the first target game replaces this public hash path by a shared oracle $H : \text{Bytes}^* \rightarrow R_q$. It does not replace the private SHAKE or ChaCha expansion at the same time. Their comparisons have separate entries in the security ledger [10, 20].

Fixed nonce length is necessary to bind (r, m) to the concatenated input. If arbitrary lengths were allowed, writing $r = u \parallel a$ and $m' = a \parallel m$ would give $r \parallel m = u \parallel m'$ with a different message. M0 requires exactly 40 bytes before calling the library. The current historical CLI's variable external-nonce interface is therefore not interchangeable with M0 [10, 11].

3.4 Trapdoor preparation and Fourier sampling

The secret polynomials determine the basis $\mathcal{B}$ and its Gram data. Fourier-domain operations compute the corresponding LDL structure and scalar sampling widths. At a two-block LDL step with $G = L \text{diag}(d_0, d_1) L^*$, sampling proceeds in reverse conditional order: the second block is sampled first, then its residual adjusts the center of the first. Mixed-radix splitting passes the problem to smaller subproblems until scalar samplers are reached.

The following equations explain the exact-algebra sampling target:

$$t = (-cF/q, cf/q), \quad y \leftarrow \text{RecursiveSample}(t, \mathcal{B}, \sigma), \quad z = (t - y)\mathcal{B}.$$

The routine is intended to yield a short pair with $z_1 + hz_2 = c$. In actual execution, FPEMU represents Fourier values, centers, widths, and reconstruction intermediates. Their relation to the above ideal expression is one of the signing-law obligations; an exact algebraic identity alone does not prove the law of rounded output coefficients [10, 21].

Algorithm 5 One signing attempt: source-level sampling interface

Input: Loaded secret state, canonical target c , fresh attempt PRNG**Output:** Two signed16 vectors (s_1, s_2) and the source fault state

- 1: Reset the per-attempt ternary sampler fault state
 - 2: Use the source Fourier representation of the target and secret basis
 - 3: Form the source target coordinates and recursively sample using the LDL tree
 - 4: At each scalar leaf, call the adaptive routine of Algorithm 6
 - 5: Reconstruct the two residual polynomials using the source FFT3/iFFT3 path
 - 6: Apply source integer rounding and signed16 conversion to both vectors
 - 7: **return** (s_1, s_2, fault)
-

This interface deliberately keeps the bit-exact recursive tree layout in the pinned source instead of replacing it by a new generic Gaussian routine. One complete attempt has 3072 scalar outputs. That number is not a bound on the internal rejection draws [10].

3.5 Adaptive scalar sampling

Let μ and τ be a runtime scalar center and width. The source computes

$$s = \text{floor}_{\text{FPEMU}}(\mu), \quad \hat{r} = \text{fl}(\mu - s), \quad \hat{d} = \text{fl}\left(1/(2\tau^2)\right).$$

A five-level, 512-entry CDF bank is selected using exponent coefficients derived from the same binary64 representations as the proposal tables. The nominal proposal variances are

$$\sigma_0^2 \in \{5, 20, 80, 320, 768\}.$$

The source chooses the narrowest eligible proposal whose coefficient $\hat{a}_0$ does not exceed $\hat{d}$. The last number in this set is a *proposal variance*; it is not the global signing width $\sigma = 768$ [7].

Algorithm 6 Adaptive scalar sampler: selected source control flow

Input: Source center μ , width τ , PRNG, per-attempt fault state**Output:** An integer sample, or a fault indication

- 1: **if** a fault is already set **then**
 - 2: **return** the source fault sentinel
 - 3: **end if**
 - 4: Check finite μ and positive finite τ ; set fault on failure
 - 5: Compute $s, \hat{r}, \hat{d}$ as specified above
 - 6: Check positive finite $\hat{d}$; set fault on failure
 - 7: **loop**
 - 8: $(k, \hat{a}_0) \leftarrow \text{AdaptiveProposal}(\hat{d})$
 - 9: On proposal-range failure, set fault and return
 - 10: $b \leftarrow$ one random bit
 - 11: $z \leftarrow -k$ if $b = 0$, otherwise $1 + k$
 - 12: $\delta \leftarrow \hat{r}$ if $b = 0$, otherwise $\text{fl}(1 - \hat{r})$
 - 13: $g \leftarrow \text{fl}(\hat{d} - \hat{a}_0)$; check nonnegative finite g
 - 14: $x \leftarrow \text{fl}(k^2 g + \text{fl}((2k\delta + \delta^2)\hat{d}))$
 - 15: Check nonnegative finite x ; set fault on failure
 - 16: **if** $\text{BerExp}_{\text{source}}(x)$ accepts **then**
 - 17: **return** $s + z$
 - 18: **end if**
 - 19: **end loop**
-

The displayed expression abbreviates the exact operation order in `sampler_large`; a bit-exact implementation must use that order. In exact arithmetic the correction

$$k^2 \left(\frac{1}{2\tau^2} - \frac{1}{2\sigma_0^2} \right) + \frac{2k\delta + \delta^2}{2\tau^2}$$

is nonnegative for an eligible proposal and $\delta \geq 0$. The source checks the corresponding computed values, and the Bernoulli routine approximates the required exponential acceptance. Its finite distribution, coefficient matching, and correction errors are part of the source-law analysis [7, 22].

The outstanding H3 interface includes the reachable-center condition

$$-2147483283 \leq \lfloor \mu \rfloor \leq 2147483281, \quad z \in [-365, 366],$$

which is designed to justify the machine conversion and addition. A finite center is not automatically an in-range integer. The current source has no new center guard inserted by this document [21].

3.6 Signature generation

M0 creates a fresh signing context for each query but uses the same secret key. A fresh source seed expands to the 40-byte nonce and then to up to 16 blocks of 56 bytes for attempt PRNG initialization. The source PRNG block comprises its key, IV, and counter fields. A maximum of $936 = 40 + 16 \cdot 56$ parent-stream bytes does not bound the child PRNG's rejection-sampling output [10, 23].

Algorithm 7 M0 signing call using the corrected core

Input: Secret bytes sk , complete message m , environment $\mathcal{E}$

Output: $\perp_{\text{pre}}$, $(r, \perp_{\text{post}})$, or (r, b)

```

1: Allocate and load a fresh source signer with  $sk$ 
2: if allocation, loading, or initial entropy fails then
3:   return  $\perp_{\text{pre}}$ 
4: end if
5: Generate  $r \in \text{Bytes}^{40}$  from the fresh source seed
6:  $c \leftarrow \text{HashToPoint}(r, m)$  ▷ Use  $H(r||m)$  in the ROM game
7: for  $a = 1, \dots, 16$  do
8:   Initialize an attempt PRNG with the next 56 parent-stream bytes
9:    $(s_1, s_2, \text{fault}) \leftarrow \text{Attempt}(sk, c)$ 
10:  if fault is set then
11:    return  $(r, \perp_{\text{post}})$ 
12:  end if
13:  if  $Q(s_1, s_2) \geq B$  then
14:    continue
15:  end if
16:  Encode  $s_2$  in STATIC with 4095 available data bytes
17:  if encoding returns failure then
18:    return  $(r, \perp_{\text{post}})$ 
19:  end if
20:  Prepend header  $\text{0xAA}$  to obtain payload  $b$ 
21:  return  $(r, b)$ 
22: end for
23: return  $(r, \perp_{\text{post}})$ 

```

Source casts occur *before* the norm test. Encoding is attempted once after norm acceptance; its failure does not cause a seventeenth attempt. The fault state belongs to an attempt and its detected fault

terminates the current call. A new persistent-latch design is not silently imported from a diagnostic branch [10].

The M0 capacity theorem excludes a lack-of-space failure at the indicated encoding step when its premises hold. It does not eliminate entropy, loader, fault, or retry failures. Nor does it prove that every positive source signing output is verified successfully: that depends on the still-separate pre-cast and signing-correctness analysis.

3.7 Signature verification

Verification is deterministic once the public target c is fixed. Public-key loading obtains canonical h and prepares the NTT/Montgomery representation. The parser selects NONE or STATIC according to the header and returns the actual signed16 coefficients s . The normalization $\rho(s)$ is used only for modular multiplication [14, 24, 25].

Algorithm 8 M0 verification through the byte-oriented corrected core

Input: Public bytes pk , message m , nonce r , payload b

Output: Accept or reject

- 1: **if** $|r| \neq 40$ or required API objects are invalid **then**
 - 2: **return** reject
 - 3: **end if**
 - 4: Load the FT1536 public-key profile; reject on failure
 - 5: $c \leftarrow \text{HashToPoint}(r, m)$ ▷ Shared $H(r||m)$ in ROM
 - 6: Check payload length, header, profile bits, and compression mode
 - 7: Decode exactly N signed16 coefficients s ; reject on decoding failure
 - 8: Require the decoder's consumed length to equal the entire data payload
 - 9: $u \leftarrow \rho(s)$ coefficientwise
 - 10: $p \leftarrow \text{Mul}_\Phi(h, u)$ ▷ Implemented using the proved NTT pipeline
 - 11: $d \leftarrow (p - c) \bmod q$ coefficientwise
 - 12: $a \leftarrow \text{center}_q(d)$ coefficientwise
 - 13: **return** accept exactly when $Q(a, s) < B$
-

The final source bound test uses a signed64 accumulator with exact products in the established domain. The first vector is $\text{center}_q(hs - c)$, whereas the extractor uses $\text{center}_q(c - hs)$. Antisymmetry of centering and $Q_0(-a) = Q_0(a)$ justify this sign difference. The modular map is computed on normalized coefficients, but the norm retains the original signed s .

3.8 Functional correctness and the ideal construction

Under exact NTRU identities and exact reconstruction, the pair generated from $(t - y)\mathcal{B}$ satisfies the public relation. If it also obeys the norm bound and is encoded without loss, the mathematical verification predicate accepts. This is the algebraic correctness argument.

For the selected implementation, two different statements must still be kept apart. Verifier soundness is already established for the pinned model: every accepted payload gives a correct short witness. Honest-signature correctness additionally requires showing that the source's rounding, narrowing, and serialization preserve the required relation on its emitted outputs. This latter statement is not obtained by reversing verifier soundness. The M0 game records positive source output as output, not as failure merely because a later verifier would reject it [9, 10].

Encoding Formats and Protocol Parameters

4.1 Records, headers, and byte lengths

M0 represents a signature by a fixed-length nonce and a variable-length payload. In a flat record, the first 40 bytes are the nonce and all remaining bytes are the payload. The record length is supplied by the enclosing transport. The payload’s first byte is its own header and is included in every payload length in this chapter [10].

Object	Header	Length	Meaning
Emitted public key	0x8A	2881	Header and 1536 packed 15-bit residues
Honest STATIC payload	0xAA	Variable	Header and compressed signed coefficients
Accepted NONE payload	0x8A	3073	Header and 1536 two-byte values, subject to decoder checks
M0 nonce	None	40	Separate from the payload
M0 caller allocation	—	4096	Payload buffer capacity, not fixed signature size
M0 maximum after norm acceptance	—	3160	Established sufficient bound, including header

Table 4.1: Wire and allocation quantities. Emitted public-key length is distinguished from all strings accepted by the loader [15, 25, 26].

For the selected profile, the payload header encodes `ternary=1`, `logn=10`, the compression selector, and a reserved bit. The only accepted compression modes are NONE and STATIC; their headers are respectively 138 and 170 in decimal. The signature parser rejects trailing data not consumed by the selected decoder [24].

The public-key loader has a different trailing-byte behavior: its decoder reads the polynomial’s 2880 data bytes but returns the full supplied length. Consequently, extra unread public-key bytes can be accepted. This is part of the modeled legacy interface, not a claim that every accepted public-key encoding is canonical. A future canonical wrapper must be specified separately rather than retroactively imposed on the current loader [25, 27].

4.2 Public and private keys

The public polynomial is packed into 15 bits per coefficient, with each decoded coefficient checked to be below 18433. The $1536 \cdot 15 = 23040$ data bits occupy 2880 bytes; adding the profile header gives 2881 bytes for the emitted public key.

The source KeyGen emits all four private polynomials in order f, g, F, G using the selected small-vector encoder. For STATIC this gives header 0xAA, followed by four separately encoded vectors. Their companions need not be ternary. The API allocation bound of 12289 bytes is not an assertion that each emitted compressed private key has that length [10, 26].

Other loader-supported variants, such as representations that omit a recoverable companion, do not redefine the selected KeyGen output format. A concrete security experiment fixes the output law of the specified generator and gives the adversary its emitted public bytes.

4.3 STATIC encoding

The active parameter is $j = 8$. An integer coefficient x is encoded as one sign bit, eight low magnitude bits, a unary run of $\lfloor |x|/256 \rfloor$ zero bits, and a terminating one bit. The bit stream is byte-packed in the source order and ends with zero padding. This describes the encoder's output language; the decoder's accepted language may contain additional representations [15].

Algorithm 9 Bit-level STATIC encoding with $j = 8$

Input: Signed16 coefficients $s_0, \dots, s_{N-1}$ and available data capacity

Output: Compressed data, or insufficient-capacity failure

- 1: Initialize an empty output bit stream
 - 2: **for** $i = 0, \dots, N - 1$ **do**
 - 3: $v \leftarrow |s_i|$, evaluated after promotion to a sufficiently wide signed type
 - 4: Emit one sign bit, equal to 1 iff $s_i < 0$
 - 5: Emit the 8 low bits of v in source order
 - 6: Emit $\lfloor v/256 \rfloor$ zero bits, then one terminating bit
 - 7: Apply the source byte-drain and capacity checks
 - 8: **end for**
 - 9: Pad the final partial byte with zeros, with its capacity check
 - 10: **return** the data bytes ▷ The signing caller adds the header
-

Its data-bit length is exactly

$$\ell_{\text{bits}}(s) = 10N + \sum_{i=0}^{N-1} \left\lfloor \frac{|s_i|}{256} \right\rfloor.$$

Thus the payload length is

$$\ell_{\text{payload}}(s) = 1 + \left\lceil \frac{\ell_{\text{bits}}(s)}{8} \right\rceil. \quad (4.1)$$

The extra byte is the header. The 40-byte nonce is excluded. All-zero coefficients give a 1921-byte payload, illustrating that neither the allocation nor the upper bound is a fixed signature length.

4.4 Accepted STATIC and NONE decoding

For STATIC, the source decoder maintains a byte cursor, a 32-bit unsigned accumulator, and a bit count. Its unary counter is also unsigned32; the range check is performed after the terminating bit, rather than imposing an earlier fixed payload limit. A formal model must therefore account for finite runs long enough to wrap that counter. Signed narrowing of the reconstructed magnitude uses the declared GCC convention, including magnitudes in [32768, 65535] and the negative-zero behavior [27].

These semantics should not be silently replaced with the mathematically simpler decoder “reject as soon as the run exceeds 255.” Both can agree on honest outputs while accepting different adversarial strings. The established L_V proof covers the implemented decoder, its termination for legal finite inputs, its guards, and all output stores.

For NONE, the decoder reads two-byte big-endian values and applies the source extension and range tests. Its successful outputs are centered coefficients, but this does not constrain STATIC outputs to that same interval. Both decoders must return exactly N coefficients and the required consumed length. The outer parser treats failure code 0 separately from a successful nonempty data stream [24, 27].

4.5 A sufficient bound on honest payload capacity

Theorem 4.1 (STATIC capacity after source norm acceptance). *For signed16 vectors s_1, s_2 in the selected FT1536 profile, suppose the defined source norm test accepts, so that $Q(s_1, s_2) < 2093922385$. Then the STATIC payload for s_2 , including its header and excluding the nonce, is at most 3160 bytes. In particular, a legal 4096-byte payload buffer is sufficient for this encoding step.*

The repository establishes this as an analytical and Lean count-model result with a pinned C correspondence. It covers length and capacity control flow, not a new proof of every encoded byte’s value [15, 23].

The argument is driven by the actual A_2 norm. Its nonnegativity gives

$$\sum_i s_{2,i}^2 \leq 2Q_0(s_2) \leq 2(B - 1) = 4187844768.$$

An integer form of Cauchy–Schwarz yields $\sum_i |s_{2,i}| \leq 2536243$, hence the sum of the unary quotients is at most 9907. Equation (4.1) then gives

$$\ell_{\text{payload}} \leq 1 + \left\lceil \frac{15360 + 9907}{8} \right\rceil = 3160.$$

Appendix A gives the integer majorant used to obtain the same endpoint without real-number square roots in the Lean count model.

4.5.1 A lower example for the capacity requirement

Let $s_1 = 0$ and $s_2 = (x, -x)$, where x has 330 coefficients equal to 1792 and 438 equal to 1536. Then

$$Q(s_1, s_2) = 330 \cdot 1792^2 + 438 \cdot 1536^2 = 2093088768 < B.$$

Its bit length is 25236, requiring 3155 data bytes and a 3156-byte payload. The archived native checks report failure at capacities 2049, 3073, and 3155, and successful encoding and roundtrip at 3156 and 4096 [15].

This is a synthetic norm-accepted vector, not a signature observed from KeyGen/Sign, an adversarial forgery, or a proof that 3160 is optimal. It is sufficient to show why the smaller historical buffer does not follow from the norm constraint alone.

4.6 Transport and integration requirements

An M0 implementation must enforce the 40-byte nonce gate and supply the selected honest capacity. The integrated Extra/c still contains the historical CLI’s 2049-byte signing buffer and variable external nonce length. The corrected verifier’s integration does not implement those caller changes [2, 11].

Under M0, the established bound corresponds to at most 3200 bytes for nonce plus payload after the stated norm-acceptance and encoding premises. This is a derived sufficient bound, not a measured mean signature size. An adversary’s accepted record can be longer because the verification language is not capped at the honest allocation. Resource accounting must therefore include the actual input byte length.

Chapter 5

Security Analysis

5.1 The target experiment

The first target is ordinary existential unforgeability under adaptive chosen-message attack in a classical, direct-output random-oracle model. One pair is sampled from $\mathcal{K}_{\text{seed}}[\mathcal{E}]$ conditional on successful KeyGen. The adversary receives pk and makes sequential signing and hash queries. It finally outputs (m^*, r^*, b^*) and wins only if m^* was never submitted to Sign, $|r^*| = 40$, the lengths are legal, and verification accepts [10].

An aborted signing query still records its message as queried. Responses distinguish a failure before nonce creation from $(r, \perp_{\text{post}})$ after nonce creation. They do not reveal a fault reason, partial output buffer, retry count, or timing. Inner loops are not given an artificial timeout: nontermination yields no final forgery. Undefined arithmetic is an unresolved source obligation, not a newly permitted failure response.

The model has resources (Q_s, Q_H, t, w, L) for signing queries, adversarial hash queries, bit-operation work, memory, and interface byte volume. The experiment is functional: QROM reprogramming and timing/physical-channel guarantees require separate arguments. These parameters matter separately: a short algebraic witness can arrive through a much longer accepted byte encoding. The direct-output oracle and the source public SHAKE/HashToPoint function are separate games to be related, not two names for the same object [10, 20].

5.2 The natural MT-ISIS relation

For the target reduction, sample h according to the public pushforward of the same successful-key law, and sample T independent uniform targets $c_1, \dots, c_T \in R_q$. A solver receives the public key and the indexed target list and must output j, z_1, z_2 such that

$$z_1 + h z_2 = c_j \text{ in } R_q, \quad Q(z_1, z_2) < B.$$

The witnesses are integer coefficient vectors; z_2 is not required to be centered. The goal uses $T = Q_H + 1$ in the proposed classical simulator. It does not automatically add another factor T for target guessing when the hardness game already provides the full list [10, 20].

We denote a hypothetical concrete upper bound on the solver's success by $\epsilon_{\text{MT}}(T; t_B, w_B)$. Obtaining or justifying that bound for this structured ring and this key law is a cryptographic assumption and a cryptanalytic task. Neither the verifier theorem nor an estimator run proves it. Work on Falcon's unforgeability also distinguishes multi-target inhomogeneous relations from second-preimage variants for stronger notions; those results concern their own schemes and modifications, not an automatic theorem for FT1536 [28].

5.3 Established verifier-to-witness soundness

Theorem 5.1 (Pinned-model byte-level extraction). *For the corrected verifier model, every canonical h, c and every legal finite payload b satisfy*

$$V_{\text{CAND}}(h, c, b) = 1 \implies \exists(z_1, z_2) = \text{Ext}_0(h, c, b) : \quad A_h(z_1, z_2) = c, \quad Q(z_1, z_2) < B,$$

where $\text{Ext}_0(h, c, b) = (\text{center}_q(c - hs(b)), s(b))$ and $s(b)$ is the parser's actual signed16 output.

The theorem is represented by L_V_BYTES, with loader and output-store variants. The premises do not assume that the byte string is honest, centered, uniquely encoded, or no longer than the honest signing capacity. The model is bound to GCC 14.2.0/C99/Linux x86_64 LP64 and its stated memory and lifecycle conditions [12, 14, 24].

The proof connects four layers. Canonical normalization supplies legal inputs for NTT. The complete transform result supplies the coefficient product rather than only an inverse transform. Centring and the exact norm computation identify the actual checked pair with the extractor up to a whole-vector sign. Finally, decoder and loader results connect successful byte parsing to those arithmetic premises. The mathematical relation and the norm are derived from the execution model rather than used to define its acceptance.

An earlier S17 verifier did not satisfy this claim for the same extractor. Its single-add normalization could misrepresent a decoded coefficient modulo q . The source correction is isolated in the verifier; the old counterexample is preserved and rejected by the corrected candidate. This is a correction of a reduction interface, not a demonstrated efficient EUF-CMA forgery [8, 11].

5.4 Signing distributions and image estimates

For $Z \leftarrow D^B$, define the public image law and its deviation from uniform:

$$P_h^B(c) = \Pr[A_h(Z) = c], \quad E_h^B = q^N \sum_c P_h^B(c)^2 - 1.$$

Writing U for uniform on R_q , this is exactly $\chi^2(P_h^B \| U)$. For positive-mass targets, let $K_{h,c} = \mathcal{L}(Z \mid A_h(Z) = c)$. If a target has no mass under the reference distribution, the ideal interface must specify a response; M0 records an explicit failure kernel instead of pretending such targets cannot occur [10].

The research history contains two distinct analytic results. T2C3 concerns one fixed, canonical public instance and its sharply truncated image. Its recorded final bound is

$$E_{h^*}^B < \frac{4489}{2^{54}(2^{24} - 1)^2} < 2^{-89}.$$

T5 concerns every successful output of the specified KeyGen, but its graph-dual theta estimate is pure/untruncated and bounded by 2^{-40} . The different quantifiers and truncations are material; T5 is not, by itself, a population version of T2C3 [2, 29].

A separate R5T argument targets a sharper truncated population estimate, with a conditional accepted-image endpoint 2^{-106} . The post-M0 record requires a consistent package and consumption of its actual interfaces before using that number in a final reduction. This document therefore does not elevate that endpoint to a completed end-to-end guarantee [2, 29].

5.5 From scalar approximation to the whole signing oracle

A signing proof must compare complete joint distributions, not just individual scalar summaries. The relevant chain includes reachable centers and widths, finite proposal laws, Bernoulli acceptance,

recursive conditional sampling, residual reconstruction, integer rounding, narrowing, norm rejection, and serialization. A good marginal bound on the transmitted component does not automatically determine the joint law before casts [20, 21].

The repository distinguishes several reference layers, including finite proposals with source acceptance, finite proposals with exact acceptance, infinite proposals with exact acceptance, ideal coset sampling with 16 norm attempts, and sampling conditioned on norm acceptance. A statement about the exact dyadic value of a computed scalar exponent coefficient is not yet a statement about the ideal global width $\sigma = 768$.

For repeated queries, bounds need to hold conditionally on the histories actually used by the hybrid argument. The shared key is retained across all queries; it cannot be redrawn to simplify each comparison. Source failures must preserve what the adversary observes. In particular, a source output later found invalid cannot simply be reclassified as an abort when the program returned bytes [10, 20].

5.6 A direct success-event transfer

An immediate conversion from χ^2 to total variation gives

$$\text{TV}(P_h^B, U) \leq \frac{1}{2} \sqrt{E_h^B}.$$

Accumulating this loss per query may be much weaker than keeping a second-moment control until the final success event. We give the alternative argument explicitly. The lemma is general probability theory, not a completed instantiation of all FT1536 game transitions. The direction of the compared laws is essential [29, 30].

Lemma 5.2 (Adaptive composition of conditional second moments). *Let R and S be transcript laws with the same initial law and successive conditional kernels R_i, S_i . Suppose, for the histories relevant to the comparison, $S_i \ll R_i$ and $\chi^2(S_i \| R_i) \leq e_i$. Then*

$$\chi^2(S \| R) \leq \prod_{i=1}^n (1 + e_i) - 1.$$

In particular, a uniform bound e gives $\Delta = (1 + e)^n - 1$.

Proof. Let $L_i = dS_{\leq i}/dR_{\leq i}$ and $\lambda_i = dS_i/dR_i$ conditional on the past. Then $L_i = L_{i-1}\lambda_i$, while $\mathbb{E}_R[\lambda_i^2 \mid \text{past}] \leq 1 + e_i$. Consequently $\mathbb{E}_R[L_i^2 \mid \text{past}] \leq (1 + e_i)L_{i-1}^2$. Iterated expectation yields the product bound, using $L_0 = 1$ and $\chi^2(S \| R) = \mathbb{E}_R[L_n^2] - 1$. Deterministic common steps can be assigned $e_i = 0$. $\square$

Lemma 5.3 (Correctly oriented event bound). *If $S \ll R$ and $\chi^2(S \| R) \leq \Delta$, let $p = R(A)$ and $r = S(A)$ for an event A . Then*

$$(p - r)^2 \leq \Delta p(1 - p), \tag{5.1}$$

$$p \leq \Phi_\Delta(r) := \frac{2r + \Delta + \sqrt{\Delta^2 + 4\Delta r(1 - r)}}{2(1 + \Delta)} \tag{5.2}$$

$$\leq \min\{1, (\sqrt{r} + \sqrt{\Delta})^2\}. \tag{5.3}$$

Proof. Put $L = dS/dR$. Since $\mathbb{E}_R(L - 1) = 0$,

$$r - p = \mathbb{E}_R[(L - 1)(\mathbf{1}_A - p)].$$

Cauchy–Schwarz bounds the square by $\mathbb{E}_R[(L - 1)^2]p(1 - p)$. Solving the resulting quadratic inequality in p gives its upper root $\Phi_\Delta(r)$. Also $p - r \leq \sqrt{\Delta p}$, and the positive-root bound on $\sqrt{p}$ is at most $\sqrt{r} + \sqrt{\Delta}$. $\square$

In the intended image comparison, the enlarged one-step laws have the form

$$R_i(c, z) = U(c)K_{h,c}(z), \quad S_i(c, z) = P_h^B(c)K_{h,c}(z).$$

With truly common conditional kernels, their χ^2 divergence is E_h^B . Common postprocessing does not increase it. Establishing these identities for the full oracle responses, including failure and history behavior, is the separate M5 task [10, 29].

5.6.1 Illustration of the difference in bounds

For hypothetical $e = 2^{-106}$, $n = 2^{20}$, and simulated success $r \leq 2^{-128}$,

$$\frac{n}{2}\sqrt{e} = 2^{-34}, \quad \Delta = (1 + e)^n - 1 \approx 2^{-86}, \quad \Phi_\Delta(r) \approx 2^{-86}.$$

These compare two analyses of one distributional transition. They are not competing measurements of FT1536's security level. Other game transitions and their ordering remain to be supplied. At $r = 0$, the exact event bound is $p \leq \Delta/(1 + \Delta)$, showing that an image-only interface still needs an appropriately small error budget for a desired concrete target.

5.7 The reduction architecture

The intended classical simulation proceeds through the following logical sequence. First, bind the real seeded primitives and source signing behavior to the specified reference laws with explicit errors and costs. Second, use a public sampler for D^B to sample a witness and program the corresponding fresh hash target. Third, analyze the difference between that reverse-sampled image and a uniform target using the chosen metric. Fourth, obtain a witness from a fresh-message accepted forgery using Theorem 5.1 [10, 20].

A simulator must account for nonce collisions, adversarial prequeries at a point it intends to program, empty reference support, retry exhaustion, and its own work caps. Once a comparison has moved to independent uniform 320-bit nonces, standard counting supplies collision and prequery bounds. Those bounds do not retroactively prove that the source's seeded nonce stream is uniform. Costs already included in a comparison are not charged a second time under another name.

The target is a bound on $\text{Adv}^{\text{EUF-CMA}}$ expressed through ϵ_{MT} and proved transition interfaces. No single completed numerical right-hand side is asserted in this version. A final expression must preserve the order of additive errors, likelihood multipliers, and the nonlinear map Φ_Δ ; commuting or adding them mechanically can invalidate an otherwise correct local analysis [20, 29].

5.8 Cryptanalytic questions and parameter assessment

Concrete assessment must preserve q , the ring polynomial, the paired metric, the successful ternary-key law, and the number of targets. Relevant questions include recovery of f, g or another usable trapdoor; direct short-preimage search; subfield or norm-based shortcuts; signing-distribution leakage; and implementation observations outside the functional game. These are distinct tasks, not interchangeable estimates [20, 29].

A larger degree or a small coefficient alphabet alone is not a security estimate. A small ideal-image deviation does not prove hard preimage search, and a verifier theorem does not prove basis-independent sampling. This draft therefore assigns no concrete security level. A future claim must combine a justified hardness assessment for the actual structured distribution with a nontrivial reduction bound at a stated resource budget.

Chapter 6

Implementation and Evaluation

6.1 Reference source and build selection

The integrated core at commit 2959064e8132443649de50600b20bfc32ed618cb contains the exact 17-file corrected candidate used by the completed normalization, NTT, L_V, and M0 checkpoints. The documentation snapshot f5266765c5f0733fb3ab6a5e2906aa44d15e17f7 adds the post-M0 freeze. The verifier file has SHA-256

3fe78f8df8003b760a21f4897b44b876717e30029bed031ee7d0cd224e968d42.

The complete candidate manifest is identified in Appendix B. This source identity matters more than a mutable branch name [2, 11].

The top-level build commands are

```
make FT1536
make check-FT1536
```

They verify the source manifest, build an out-of-tree copy, and record compiler information and output hashes. The focused check includes finite FPEMU checks, the CLI usage path, and rejection of the historical verifier witness. It builds the historical test executable but does not run its full KeyGen/Sign benchmark as part of that focused check [1, 11].

6.2 FPEMU and the scalar tables

The implementation selects `fpr-emulated.h` and links `fpr-emulated.o`. Its headers preserve attribution to the Falcon FPEMU lineage; FT1536 adds the ternary adaptations and the current sampling profile. The active scalar table bank has five levels and 512 entries per level [1, 7].

The compiler profile uses `-O`, the adaptive CDF path, CT-BerExp, the full-ternary proposal mode, and the fixed 16-attempt signing cap. Those choices are part of the research configuration. An alternative optimizer, floating-point backend, table bank, or attempt limit is not the same source profile even when ordinary tests pass [6].

FPEMU’s role is numerical representation and controlled operation behavior. It does not turn every formula involving its outputs into an exact-real computation. The numerical proof must track the computed centers and widths, their finite representations, and how errors propagate through the recursive sampler and reconstruction.

6.3 Modular verification pipeline

The public-key loader precomputes the NTT and Montgomery representation of h . The signed response is normalized to ordinary canonical residues before its own forward transform. Pointwise multiplication and inverse transformation yield the polynomial product, followed by modular subtraction of the target. The established theorem is

$$\text{pipelineC}(h, r, c) = (\text{product}(h, r), \text{subtract}(\text{product}(h, r), c))$$

for canonical inputs. The product on the right is independently defined through coefficient multiplication and ring reduction [16].

The full source model covers temporary values, initialized table indices, prefix invariants for in-place stores, and reverse traversal. This is stronger than comparing only a forward/inverse pair on a finite vector set. It supplies the exact interface consumed by the byte-verifier proof.

6.4 Space and size results

The current size information is summarized below. A bound is not a measured average, and a caller capacity is not a wire length.

Quantity	Bytes	Evidence class
Emitted public key	2881	Format calculation and source encoding
NONE payload	3073	Format and decoder contract
Zero-vector STATIC payload	1921	Exact bit-count calculation
Synthetic short STATIC example	3156	Exact example and archived native controls
STATIC upper bound after norm acceptance	3160	Analytical and formal count-model bound
Nonce plus that upper bound	3200	Derived 40 + 3160
Selected M0 payload capacity	4096	Caller choice with proved sufficiency

Table 6.1: Size quantities for the current profile [10, 15, 25].

The public-key decoder accepts some longer encodings, and the verification input language is not limited by the honest STATIC bound. Buffer management and adversarial parsing cost therefore require their own limits at the application layer, without changing the theorem’s input domain unnoticed.

6.5 Reported validation results

The archive distinguishes formal proof checks from execution controls. Full L_NTT has a reported replay of 217/217 semantic files; the byte-level L_V replay reports 402/402; M0 reports 273/273. The L_V dependency closure has 73 modules and 570 theorems, while M0’s selected closure has 67 modules and 535 theorems. These closures reuse results and must not be added to claim a number of distinct project theorems [1, 23, 31].

L_V’s recorded native controls include 55 byte cases, 8 norm cases, and 4 centering cases, with normal and ASan/UBSan builds. In both encodings, real vectors at $B - 1$, B , $B + 1$ yield decisions 1, 0, 0. Mutations of the center boundary, paired offset, cross-term sign, strict comparison, and narrowing are detected. M0 separately records capacity, framing, and contract-consistency controls [23, 31].

These finite tests help assess the source-model binding and the behavior of executable checkers. They do not supply the universal quantifiers in the formal theorems. Conversely, an axiom audit alone does not show that a model describes the intended C program; correspondence, exact inputs, and the meaning of the final theorem still require review [27, 32].

6.6 Performance reporting policy

This version does not supply an invented performance comparison with Falcon-512 or Falcon-1024. The material reviewed for the current full-ternary-secret profile does not establish a comparable throughput and memory table for the exact M0 caller and integrated core. Earlier ternary checkpoints used different parameter or secret profiles and are not silently reused as current measurements.

A reproducible performance evaluation should identify the CPU, compiler and flags, core commit, caller contract, random-input procedure, repetition count, and measured operation boundaries. KeyGen

should report both completion probability and work per completed call. Sign should include failed calls and distinguish per-call latency from throughput conditional on acceptance. Verification should include the actual byte lengths and the selected encoding. Memory measurements should separate persistent key state, working buffers, and caller allocations.

Operation	Measurement required	Status in this manuscript
KeyGen	Success frequency, time distribution, peak memory	No current-profile benchmark asserted
Sign	Successful and failed-call latency, payload distribution	No current-profile benchmark asserted
Verify	Time by encoding and payload length	No current-profile benchmark asserted
Proof replay	Exact checkpoint, environment, reproduced outputs	Repository receipts reported separately

Table 6.2: Performance information deliberately separated from the established mathematical size results.

6.7 Reproducibility and the trust boundary

The recorded proof environment is Lean 4.34.0/Std, SageMath 10.9, and GCC 14.2.0 under Linux x86_64 LP64. The repository’s replay scripts also pin paths and inputs. A reviewer using a different setup should document that configuration instead of presenting it as an identical replay [31, 33].

```
python3 -B proofs/ft1536/tools/archive.py verify
python3 -B -m unittest discover -s proofs/ft1536/tests -v
python3 -B proofs/ft1536/tools/archive.py replay \
  FT1536_M0_CONTRACT_RUN_001 --run review-001 \
  --timeout 1200 --hide-originals
```

Archive verification checks file identity and declared inputs. Replay rebuilds the specified computations in a fresh location. The formal kernel checks the Lean models. The explicit C translation and ABI assumptions connect those models to the implementation. No verified compiler is claimed, and the manuscript preparation itself did not rerun the project toolchain. The companion validation script checks only selected arithmetic statements used in the exposition.

Research Status and Further Development

7.1 Results established at this snapshot

FT1536 now has a source-bound chain from normalization to full modular multiplication and from accepted bytes to a short lattice witness for the corrected verifier model. The corrected core has been integrated into the default build. Separately, M0 fixes an explicit protocol/game contract and proves sufficient STATIC capacity after defined norm acceptance [2, 11, 23].

These are complementary interfaces: algebra specifies the witness, verifier soundness derives it from acceptance, and the capacity result justifies an allocation. Honest-output distributions and computational hardness remain different questions.

7.2 The remaining dependency structure

The next source-level interface is reachable scalar arithmetic: emitted keys and reachable signing prefixes must satisfy H3's center and conversion conditions. This connects to the joint Fourier-sampling law, pre-cast behavior, and complete observable signing responses. The population image estimate and the general χ^2 event-transfer lemma can be developed separately, but their use in a simulator requires the same games, kernels, and byte processing [21, 29].

A final classical reduction must combine these components with public-sampler resources, oracle programming, primitive assumptions, and the appropriate MT-ISIS instance distribution. A concrete security claim additionally requires a nontrivial numerical bound at a stated resource budget. QROM and implementation side-channel guarantees remain separate extensions.

7.3 Specification changes versus proof extensions

A proof extension may close an obligation without changing the code. By contrast, adding a new guard, a different table bank, a new cutoff, or a different retry policy creates a new source or distribution profile. Such a change needs a new identity and a review of all consumed assumptions. Even a correction that preserves the public verification equation can change the law of successful keys or signatures.

The intended evolution of this document is therefore additive where possible: fill in proved interfaces and measured performance for the same selected profile; create a new version when semantics change. Historical counterexamples and earlier partial results remain valid statements about their original source versions.

7.4 Conclusion

The construction is specified from its algebraic objects to its byte interfaces. The corrected verifier's extraction theorem is the main confirmed security interface. The remaining analytical work connects actual signing behavior to reference distributions and a concrete unforgeability bound.

Chapter A

Selected Proofs and Consistency Arguments

A.1 Canonical normalization on the full signed16 domain

The corrected helper computes

$$\rho_C(x) = (x + 36866) \% 18433.$$

For $-32768 \leq x \leq 32767$, its dividend lies in $[4098, 69633]$. Thus the signed32 addition is exact, the dividend is positive, the remainder is nonnegative, and the final unsigned16 conversion is exact. Adding $2q$ preserves the residue, so $\rho_C(x) = \rho(x)$. This is the elementary arithmetic behind `rho_contract`; the source-bound checkpoint also covers the vector loop and the precise range on which the old map was already canonical [13, 34].

The old model was

$$\iota(x) = (x + q \mathbf{1}_{x < 0}) \bmod 65536.$$

For $x = -20000$, $\iota(x) = 63969$, while $\rho(x) = 16866$. The residues of 63969 and -20000 modulo q differ because the word modulus 65536 is not divisible by q .

The recorded S17 example uses a fixed public h^* and a synthetic target such that the old source verifier accepts with checked norm 400000000, while the specified extractor has norm $43058711057 > B$. This refutes that extraction implication for S17. It does not supply a preimage of the actual message hash. The integrated corrected core rejects the same witness [8, 11].

A.2 Verifier soundness from the completed pipeline

Let s be the exact signed16 decoded response. The completed `L_NTT_rho` gives

$$p = hs \pmod{q, \Phi}, \quad d = (p - c) \bmod q.$$

The modeled source centering yields $a = \text{center}_q(d) = \text{center}_q(hs - c)$. With $z_1 = \text{center}_q(c - hs)$ and $z_2 = s$, centering antisymmetry gives $a = -z_1$. Therefore

$$Q(a, s) = Q(z_1, z_2), \quad z_1 + h z_2 = c \pmod{q, \Phi}.$$

The exact strict source norm predicate implies $Q(z_1, z_2) < B$ on acceptance. Decoder theorems supply defined s and its signed16 range; loader theorems supply canonical h and the correct prepared representation [14, 16, 24].

The proof of the norm predicate bounds each signed32 product and all signed64 accumulation prefixes. This justifies the mathematical sum of 3072 squares and 1536 cross terms without signed overflow. It also prevents replacing strict $Q < B$ with a non-strict test at the boundary [9, 27].

A.3 Integer-only derivation of the capacity endpoint

From $Q(s_1, s_2) < B$ and the A_2 identity,

$$\sum_i s_{2,i}^2 \leq 4187844768.$$

For every integer x , $(|x| - 1651)^2 \geq 0$, hence

$$3302|x| \leq x^2 + 2725801.$$

Summing the $N = 1536$ inequalities gives

$$3302 \sum_i |s_{2,i}| \leq 4187844768 + 1536 \cdot 2725801 = 8374675104.$$

Since $3302 \cdot 2536244 = 8374677688 > 8374675104$, the integer sum is at most 2536243. It follows that

$$\sum_i \lfloor |s_{2,i}|/256 \rfloor \leq \lfloor 2536243/256 \rfloor = 9907.$$

This recovers the 3160-byte upper bound. The proof concerns sufficient capacity, not the exact supremum of possible output lengths [15].

A.4 Why fixed nonce length is enough for input framing

Let $|r| = |r'| = 40$. If $r||m = r'||m'$, equality of the first 40 bytes implies $r = r'$, and equality of the remaining suffixes implies $m = m'$. Thus the concatenation map is injective on this domain. This is a framing statement; it does not assert hash injectivity or collision resistance.

With unrestricted nonce lengths the statement is false, since $(u||a)||m = u||(a||m)$. M0's gate rules out the shorter nonce in this example. The same gate must be implemented in the actual caller for the contract to apply [10].

A.5 Interpreting the loss comparison

The bound $\chi^2(S||R) \leq \Delta$ controls $\mathbb{E}_R[(dS/dR)^2]$. A direct Hölder argument in this direction bounds $S(A)$ through $R(A)$, not the reverse. Lemma 5.3 instead uses the centered event indicator and solves the quadratic inequality to bound $R(A)$ through $S(A)$.

At $r = 0$, the upper endpoint is $\Delta/(1 + \Delta)$ and is attainable by an appropriate Bernoulli example. Thus even this sharper universal interface does not make an arbitrary image error free. Better parameters, additional structure, or another justified comparison can be relevant, but they require an additional argument rather than reversing the divergence symbol.

Chapter B

Artifact Map and Manuscript Boundaries

B.1 Source identity

The manuscript is based on documentation snapshot

f5266765c5f0733fb3ab6a5e2906aa44d15e17f7

and the integrated core at

2959064e8132443649de50600b20bfc32ed618cb

in niirmataa/free_falcon_sign. The authoritative 17-file candidate manifest has SHA-256

2553358fbb1144acdb577e1ad371a017ac325306dd901af745295dd17b03bd5a.

The post-M0 freeze documents this state without converting historical execution-time flags into current deployment claims [2, 11].

B.2 Reader’s map to the evidence

The following paths are relative to proofs/ft1536/. Full pinned links are included in the bibliography and the companion source ledger.

Subject	Artifact entry point
Overall frozen state	stages/FT1536_POST_M0_FREEZE_RUN_001/REPORT.md
Normalization	stages/FT1536_L_RHO_RUN_001/formal/Rho.lean
Complete modular pipeline	stages/FT1536_L_NTT_FORWARD_RUN_001/formal/Complete.lean
Raw extraction	stages/FT1536_L_V_BRIDGE_RUN_001/formal/RawBridge.lean
Accepted bytes	stages/FT1536_L_V_BRIDGE_RUN_001/formal/VerifyBytes.lean
Source correspondence	stages/FT1536_L_V_BRIDGE_RUN_001/SOURCE_MODEL_BINDING.md
Game and observations	stages/FT1536_M0_CONTRACT_RUN_001/GAME.md
Capacity proof	stages/FT1536_M0_CONTRACT_RUN_001/CAPACITY.md
Reduction interfaces	stages/FT1536_M0_CONTRACT_RUN_001/HOP_LEDGER.md
Next source interface	stages/FT1536_M0_CONTRACT_RUN_001/H3_INTERFACE.md
Post-L_V roadmap	documents/FT1536_MAPA_DALSZYCH_DZIALAN_PO_LV_2026-09-19.md

B.3 Formal results and unproved target interfaces

The completed normalization, NTT, verifier, and capacity developments have their own exact theorem types and recorded axiom audits. The target EUF-CMA theorem is not an assumption packaged as an established result. The ledger distinguishes numerical obligations, distribution comparisons, computational assumptions, and resource bounds [20, 23].

One practical interface detail deserves attention during final composition. `L_V_LOADED` constructs its witness using the decoded public polynomial, but its exported conclusion existentially quantifies that polynomial without separately returning the equality binding it to the loaded input key. A reduction may retain the loader equation explicitly and apply the raw or byte theorem, or strengthen the exported loader result. This is a theorem-interface requirement, not evidence against `L_V_BYTES` [24, 25].

B.4 Attribution and preparation of this draft

FT1536 is credited to the repository identity Niirmata. Its implementation continues the historical Falcon EXTRA line and retains the source attribution of the Falcon contributors. The FPEMU core has its own inherited provenance. No endorsement by the original Falcon authors is implied [1, 11].

The project reports AI-assisted research and implementation using GPT-ASTRA, with a separate Codex/Daybreak Blue review of the historical counterexample. The present manuscript was also prepared with AI assistance from repository materials. Tool use is not independent cryptographic validation; the author remains responsible for reviewing the statements, references, and correspondence between pseudocode and source before publication [8, 9, 23].

Preparation of draft 0.2 involved reading the cited repository files, inspecting the organization of the Falcon specification, and revising the earlier verifier-centered manuscript into a construction-level account. It did not run the full Lean, Sage, or C replay. A small companion script checks selected integer identities, normalization counts, capacity arithmetic, and the numerical event-transfer illustration. Its scope is limited to those computations.

B.5 Editorial completion before submission

The author should approve the design rationale and title, complete any missing machine-level specifications needed for independent reimplementations, decide the precise publication scope, and obtain an independent review of the source-model boundary and cryptographic argument. A reproducible benchmark campaign should precede any throughput or comparative-efficiency claim. The integrated M0 wrapper, full signing-law theorem, final concrete reduction, and any QROM theorem must be added only when their corresponding work is complete.

This is a research specification draft, not a submission to a named venue or a declaration of standardization. The document's layout follows the genre of a cryptographic construction specification; its text, algorithms, and FT1536-specific organization are newly written rather than a reproduction of the Falcon specification.

Bibliography

- [1] Niirmata. *FREE Falcon–FT1536: repository overview*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [2] Niirmata. *Frozen post-M0 FT1536 research state*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [3] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. *Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU*. Specification v1.2, Falcon Project, 2020. [Specification PDF](#), especially Section 1.4.
- [4] Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. *Trapdoors for Hard Lattices and New Cryptographic Constructions*. Proceedings of the 40th Annual ACM Symposium on Theory of Computing, pp. 197–206, 2008. [Publication record](#).
- [5] Niirmata. *FT1536 source key generation and filtering*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [6] Niirmata. *FT1536 selected compiler and source profile*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [7] Niirmata. *FT1536 adaptive sampler and signing source*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [8] Niirmata. *FT1536: confirmed S17 extraction counterexample and Blue handoff*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [9] Niirmata. *FT1536 L_V_BRIDGE: byte-level verifier proof report*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [10] Niirmata. *FT1536 M0: exact family of security games*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [11] Niirmata. *Active FT1536 build: source integration and provenance*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [12] Niirmata. *FT1536 L_V: exact pinned-model claim*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [13] Niirmata. *FT1536: formal normalization contracts*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [14] Niirmata. *FT1536: raw verifier soundness and sign bridge*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [15] Niirmata. *FT1536 M0: STATIC capacity bound and synthetic witness*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [16] Niirmata. *FT1536: complete NTT composition and rho substitution*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.

- [17] Niirmata. *FT1536 L_NTT_GLOBAL: global inverse checkpoint*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [18] Niirmata. *FT1536 source companion conversion: poly_big_to_small*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [19] Niirmata. *FT1536 public hash and corrected verifier source*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [20] Niirmata. *FT1536 M0: security-transition ledger*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [21] Niirmata. *FT1536 M0: next source-level H3 interface*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [22] Niirmata. *FT1536 exact scalar sampler entry used by H3*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [23] Niirmata. *FT1536 M0: profile, game, and parametric ledger*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [24] Niirmata. *FT1536: byte-verifier definitions and exported theorems*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [25] Niirmata. *FT1536: public-key decoding and preparation*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [26] Niirmata. *FT1536 source key serialization*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [27] Niirmata. *FT1536: byte-verifier source/model correspondence*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [28] Pierre-Alain Fouque, Phillip Gajland, Hubert De Groote, Jonas Janneck, and Eike Kiltz. *A Closer Look at Falcon*. EUROCRYPT 2026. [IBM Research publication record and abstract](#); accessed 19 September 2026.
- [29] Niirmata. *FT1536: post-L_V roadmap and conditional chi-square transfer*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [30] Tim van Erven and Peter Harremoës. *Rényi Divergence and Kullback–Leibler Divergence*. IEEE Transactions on Information Theory, 60(7):3797–3820, 2014. DOI: 10.1109/TIT.2014.2320500. [Author manuscript](#).
- [31] Niirmata. *FT1536 L_V_BRIDGE: maintainer replay and review record*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.
- [32] Lean Project. *Validating a Lean Proof*. The Lean Language Reference. [Official documentation](#); accessed 19 September 2026.
- [33] Niirmata. *FT1536 proof archive and replay instructions*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the companion source ledger.
- [34] Niirmata. *FT1536 L_RHO: local proof and minimal candidate*. 2026. [Pinned repository artifact](#), snapshot f5266765. Exact path in the accompanying source ledger.